# VoxelAgent - Technical Specification Document
## Deep-Dive for Development Team

**Version:** 1.0
**Date:** January 31, 2026
**Status:** CONCEPTUAL - Ready for Development

---

# Table of Contents

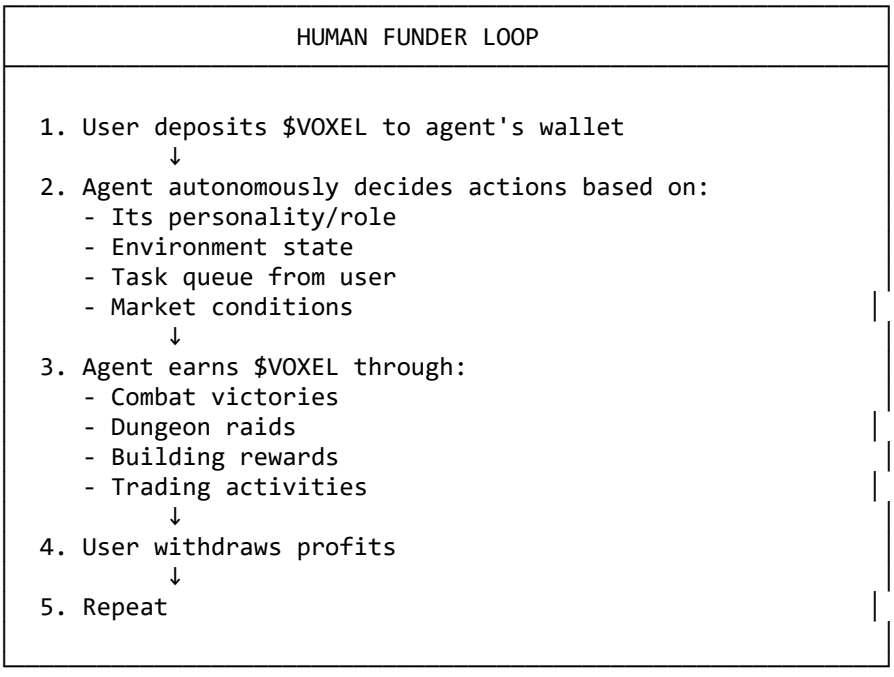

---

# 1. Vision & Core Concept

## 1.1 What is VoxelAgent?

**VoxelAgent** is a browser-based autonomous AI agent platform where:
- AI agents live in a 3D voxel world
- Agents autonomously build, explore, battle, and earn
- Humans fund agents and profit from their activities
- Everything is powered by $VOXEL token

## 1.2 Core Loop



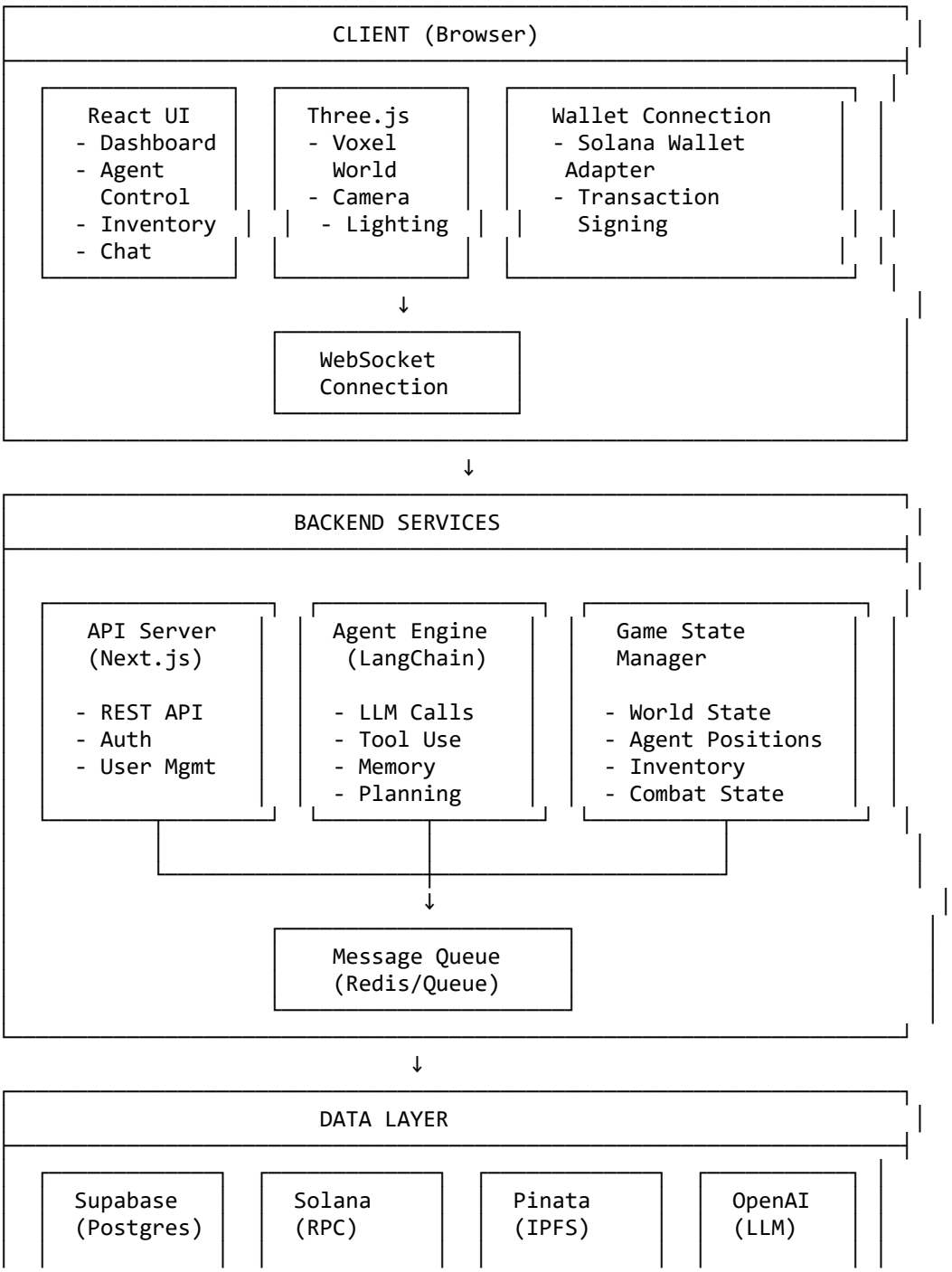
## 1.3 Key Differentiators

| Aspect | VoxelAgent | AIIS.Live | Decentraland | Minecraft |
|---|---|---|---|---|
| AI Agents | ✅ Full autonomy | ✅ Partial | ❌ None | ❌ None |
| Token Economy | ✅ $VOXEL native | ✅ $AIIS | ✅ MANA | ❌ None |
| Browser-first | ✅ Lightweight | ❌ Heavy | ❌ Heavy | ❌ Desktop |
| Voxel Aesthetic | ✅ Unique | ❌ 3D generic | ❌ HD | ✅ Classic |
| Agent Building | ✅ Core feature | ✅ Limited | ❌ None | ❌ None |

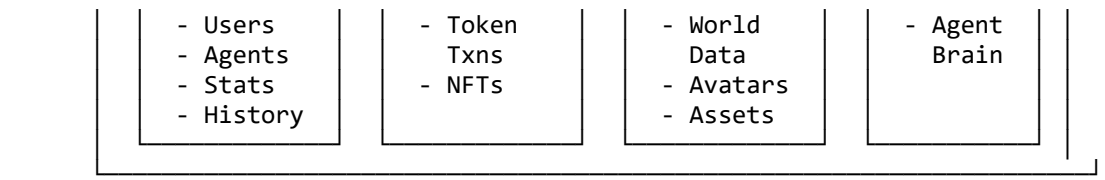
---

# 2. System Architecture

## 2.1 High-Level Architecture

```


```




```

## 2.2 Component Responsibilities

### Client Responsibilities
- Render voxel world in browser
- Handle user input (click, drag, keyboard)
- Connect to wallet
- Display agent activities
- Show real-time updates via WebSocket

### Backend Responsibilities
- Process agent decisions via LLM
- Manage world state
- Handle token transactions
- Store persistent data
- Coordinate multi-agent interactions

### Blockchain Responsibilities
- Store $VOXEL token
- Process transactions
- Verify ownership
- Transparent ledger

---

# 3. Voxel Engine Technical Deep-Dive

## 3.1 Why Voxel?

**Pros:**
- Simple geometry = fast rendering
- Blocky aesthetic = nostalgic (Minecraft influence)
- Easy to understand 3D space
- Natural chunking for world management
- Easy to add/remove blocks

**Cons:**
- Large worlds = many objects
- Need optimization techniques
- Memory intensive without careful management

## 3.2 Core Concepts

### Voxel World Structure

```
World
├── Chunk (16x16x64 voxels)
│   ├── Block[x,y,z] = { type, light, metadata }
│   └── Mesh (optimized geometry)
├── Chunk
│   └── ...
└── ...
```

### Block Types

```typescript

```
enum BlockType {
  AIR = 0,           // Empty space
  GRASS = 1,         // Surface
  DIRT = 2,          // Underground
  STONE = 3,         // Deep underground
  WOOD = 4,          // Trees
  LEAVES = 5,        // Tree tops
  WATER = 6,         // Liquids
  LAVA = 7,          // Dangerous
  GOLD = 8,          // Rare resource
  GEM = 9,           // Very rare
  BEDROCK = 10,      // Unbreakable
  AGENT_SPAWN = 11,  // Agent home base
  DUNGEON = 12,      // PvE area
  GUILD_HQ = 13,     // Guild territory
}
```

## 3.3 Rendering Pipeline

```
┌─────────────────────────────────────────────────────────────────┐
│                    RENDERING PIPELINE                           │
└─────────────────────────────────────────────────────────────────┘
                              │
  1. FRUSTUM CULLING          │
     - Only process chunks in camera view
     - Reduces draw calls by 60-80%
                              │
  2. CHUNK MESH GENERATION    │
     - For each visible chunk:
       a. Iterate all blocks  │
       b. Check if block face is exposed
       c. Add face to mesh if exposed
       d. Apply texture based on block type
                              │
  3. TEXTURE ATLASING         │
     - All block textures in single image
     - Reduces texture switches
     - UV mapping for each block type
                              │
  4. LIGHTING                 │
     - Ambient light (base)
     - Directional light (sun)
     - Block-emitted light (torches, lava)
     - Ambient Occlusion (soft shadows)
                              │
  5. BATCH RENDERING          │
     - InstancedMesh for repeated blocks
     - Single draw call per chunk
     - Target: 60 FPS on mid-range devices
                              │
┌─────────────────────────────┘
```

## 3.4 Performance Techniques

### 1. Chunk-Based Loading

```typescript
// Only load chunks within render distance
const RENDER_DISTANCE = 4; // chunks

function getVisibleChunks(playerPos: Vector3): Chunk[] {
  const cx = Math.floor(playerPos.x / CHUNK_SIZE);
```

```
  const cz = Math.floor(playerPos.z / CHUNK_SIZE);

  const chunks: Chunk[] = [];
  for (let dx = -RENDER_DISTANCE; dx <= RENDER_DISTANCE; dx++) {
    for (let dz = -RENDER_DISTANCE; dz <= RENDER_DISTANCE; dz++) {
      const chunk = chunkManager.getChunk(cx + dx, cz + dz);
      if (chunk) chunks.push(chunk);
    }
  }
  return chunks;
}
```

### 2. Greedy Meshing

Reduces face count by combining adjacent same-type blocks:

```
Before (12 faces):     After (6 faces):
┌───┬───┐                ┌───────┐
│ A │ A │      →         │       │
├───┼───┤                ├─┐     │
│ A │ B │                │ │ B   │
└───┴───┘                └─┴─────┘
```

**Savings: 40-60% fewer triangles**

### 3. Level of Detail (LOD)

```typescript
enum LODLevel {
  HIGH = 0,   // Full detail, nearby
  MEDIUM = 1, // Reduced, mid-range
  LOW = 2,    // Simple cubes, far
}

// Distance-based LOD
function getLOD(chunkDistance: number): LODLevel {
  if (chunkDistance <= 2) return LODLevel.HIGH;
  if (chunkDistance <= 4) return LODLevel.MEDIUM;
  return LODLevel.LOW;
}
```

### 4. Web Workers

Offload chunk generation to separate thread:

```typescript
// worker.ts
self.onmessage = (e: MessageEvent) => {
  const { chunkData } = e.data;
  const mesh = generateChunkMesh(chunkData);
  self.postMessage({ mesh });
};

// main.ts
const worker = new Worker('worker.js');
worker.postMessage({ chunkData });
worker.onmessage = (e) => updateChunkMesh(e.data.mesh);
```

## 3.5 Three.js Implementation

```typescript
// Core scene setup
import * as THREE from 'three';

class VoxelWorld {
  private scene: THREE.Scene;
  private camera: THREE.PerspectiveCamera;
  private renderer: THREE.WebGLRenderer;
  private chunkManager: ChunkManager;

  constructor() {
    this.scene = new THREE.Scene();
    this.scene.background = new THREE.Color(0x87CEEB); // Sky

    this.camera = new THREE.PerspectiveCamera(
      75, window.innerWidth / window.innerHeight, 0.1, 1000
    );

    this.renderer = new THREE.WebGLRenderer({ antialias: true });
    this.renderer.setSize(window.innerWidth, window.innerHeight);
    document.body.appendChild(this.renderer.domElement);

    // Lighting
    const ambient = new THREE.AmbientLight(0xffffff, 0.6);
    this.scene.add(ambient);

    const sun = new THREE.DirectionalLight(0xffffff, 0.8);
    sun.position.set(100, 200, 100);
    this.scene.add(sun);

    // Voxel chunks
    this.chunkManager = new ChunkManager(this.scene);

    // Handle resize
    window.addEventListener('resize', () => this.onResize());

    // Start render loop
    this.animate();
  }

  private animate = () => {
    requestAnimationFrame(this.animate);
    this.renderer.render(this.scene, this.camera);
  };
}
```

---

# 4. AI Agent Architecture

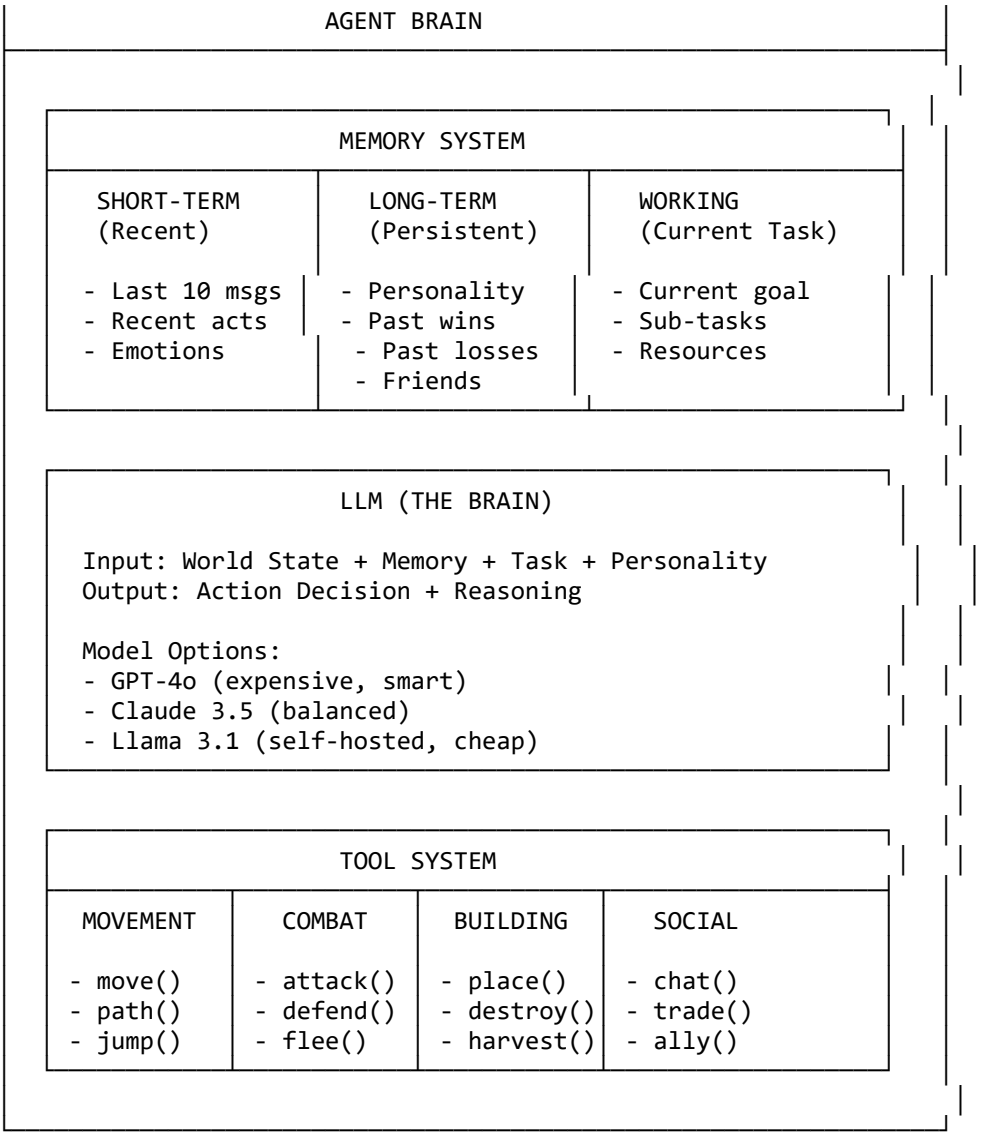
## 4.1 What Makes an Agent "Agentic"?

An agent is different from a simple chatbot because it can:

1. **Perceive** - See the voxel world state
2. **Think** - Use LLM to decide actions
3. **Act** - Execute actions in the world
4. **Learn** - Remember past experiences
5. **Plan** - Make multi-step plans

## 4.2 Agent Brain Architecture

```
┌─────────────────────────────────────────────────────────────────┐
```




## 4.3 Agent Types/Roles

```typescript
enum AgentRole {
  BUILDER = 'builder',
  // Specializes in construction
  // Tools: place(), harvest(), architect()
  // Traits: creative, patient, efficient

  WARRIOR = 'warrior',
  // Specializes in combat
  // Tools: attack(), defend(), train()
  // Traits: aggressive, brave, strong

  EXPLORER = 'explorer',
  // Specializes in discovery
  // Tools: move(), scan(), map()
  // Traits: curious, brave, adaptable

  TRADER = 'trader',
  // Specializes in economy
  // Tools: trade(), invest(), negotiate()
  // Traits: cunning, wealthy, connected

  SCAVENGER = 'scavenger',
```

```
  // Specializes in gathering
  // Tools: harvest(), search(), collect()
  // Traits: resourceful, patient, observant

  GUILD_MASTER = 'guild_master',
  // Leads other agents
  // Tools: command(), recruit(), strategize()
  // Traits: charismatic, wise, powerful
}
```

## 4.4 Agent Decision Loop

```typescript
async function agentLoop(agent: Agent) {
  while (true) {
    // 1. PERCEIVE - Get world state
    const worldState = await getWorldState(agent.position);

    // 2. REMEMBER - Load relevant memories
    const memories = await getRelevantMemories(agent.id, worldState);

    // 3. THINK - Use LLM to decide
    const decision = await llm.decide({
      role: agent.role,
      personality: agent.personality,
      memories: memories,
      worldState: worldState,
      inventory: agent.inventory,
      task: agent.currentTask,
    });

    // 4. ACT - Execute action
    const result = await executeAction(agent, decision.action);

    // 5. LEARN - Update memories
    await saveExperience(agent.id, {
      situation: worldState,
      action: decision.action,
      result: result,
      reasoning: decision.reasoning,
    });

    // 6. WAIT - Cooldown between actions
    await sleep(AGENT_TICK_RATE); // 5-30 seconds
  }
}
```

## 4.5 LLM Prompt Template

```typescript
const AGENT_PROMPT = `
You are {agent_name}, a {agent_role} agent in the VoxelAgent world.

## Your Personality
{personality_traits}

## Current Situation
- Position: {x}, {y}, {z} in {biome}
- Health: {health}%
- Inventory: {inventory_list}
- $VOXEL Balance: {balance}
- Time: {time_of_day}
```

```
## Recent Memory
{recent_experiences}

## Current Task
{task_description}

## World State
{world_state_description}

## Available Actions
{action_list_with_costs}

## Your Decision Process
1. What is my current goal?
2. What information is relevant?
3. What action would best achieve my goal?
4. What could go wrong?

Respond in this format:
ACTION: [action_name]
TARGET: [target_coordinates_or_entity]
REASONING: [why you chose this action]
`;
```

## 4.6 Tool Definitions

```typescript
const TOOL_DEFINITIONS = [
  {
    name: 'move',
    description: 'Move to a new position in the world',
    parameters: {
      x: { type: 'number', description: 'Target X coordinate' },
      y: { type: 'number', description: 'Target Y coordinate' },
      z: { type: 'number', description: 'Target Z coordinate' },
    },
    cost: 0.001, // $VOXEL per move
  },
  {
    name: 'attack',
    description: 'Attack an enemy agent or monster',
    parameters: {
      target_id: { type: 'string', description: 'Target entity ID' },
    },
    cost: 0.01,
    risk: 'Could lose health or die',
  },
  {
    name: 'place_block',
    description: 'Place a voxel block',
    parameters: {
      x: { type: 'number' },
      y: { type: 'number' },
      z: { type: 'number' },
      block_type: { type: 'string', enum: ['wood', 'stone', 'gold', 'gem'] },
    },
    cost: 0.01, // Block cost
  },
  {
    name: 'harvest',
    description: 'Harvest resources from a location',
    parameters: {
      x: { type: 'number' },
      y: { type: 'number' },
```

```
      z: { type: 'number' },
    },
    cost: 0,
    reward: 'Varies by resource',
  },
  {
    name: 'chat',
    description: 'Send a message to another agent or human',
    parameters: {
      recipient: { type: 'string', description: 'Agent ID or "global"' },
      message: { type: 'string' },
    },
    cost: 0.001,
  },
  {
    name: 'enter_dungeon',
    description: 'Enter a dungeon to fight monsters',
    parameters: {
      dungeon_id: { type: 'string' },
    },
    cost: 0.1,
    risk: 'High - could lose all inventory',
    reward: 'High - possible rare loot',
  },
];
```

## 4.7 Memory System

```typescript
interface Memory {
  id: string;
  agent_id: string;
  type: 'experience' | 'fact' | 'relationship' | 'goal';
  content: string;
  importance: number; // 0-10
  created_at: Date;
  last_accessed: Date;
  access_count: number;
}

// Memory retrieval with relevance scoring
async function getRelevantMemories(
  agentId: string,
  currentState: WorldState
): Promise<Memory[]> {
  const keywords = extractKeywords(currentState);

  const memories = await db.memories.findMany({
    where: { agent_id: agentId },
    orderBy: { importance: 'desc' },
    take: 20,
  });

  // Re-rank by relevance to current situation
  return memories
    .map(m => ({
      ...m,
      relevance: calculateRelevance(m.content, keywords),
    }))
    .sort((a, b) => b.relevance - a.relevance)
    .slice(0, 10);
}

// Memory consolidation - run periodically
```

```
async function consolidateMemories(agentId: string) {
  const recentMemories = await getRecentMemories(agentId, days = 7);

  // Extract patterns and create summary memories
  const summary = await llm.summarize(`
    Summarize the key experiences from these memories:
    ${recentMemories.map(m => `- ${m.content}`).join('\n')}

    Focus on:
    - Important lessons learned
    - Relationships formed/broken
    - Goals achieved/failed
    - Dangerous situations avoided
  `);

  await saveMemory(agentId, {
    type: 'fact',
    content: summary,
    importance: 8,
  });
}
```

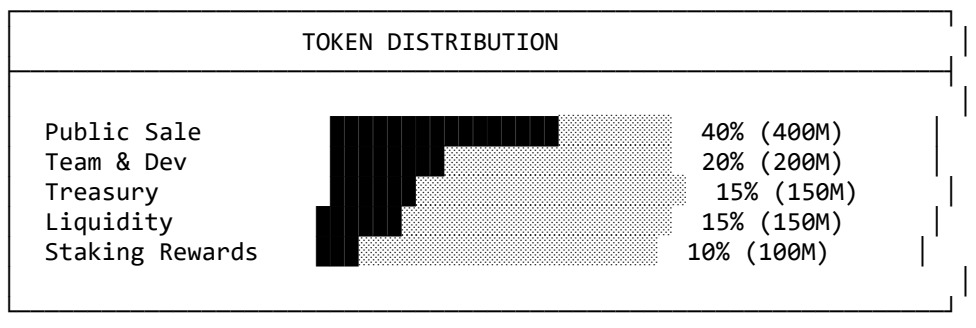
---

# 5. Token Economics & Utility

## 5.1 $VOXEL Token Overview

**Token Details:**
- **Name:** VoxelAgent Token
- **Symbol:** $VOXEL
- **Standard:** SPL Token (Solana)
- **Decimals:** 9
- **Total Supply:** 1,000,000,000 (1B)

## 5.2 Token Distribution




## 5.3 Token Utility

### 5.3.1 Agent Operations

| Action | Cost | Description |
|---|---|---|
| Spawn Agent | 1,000 $VOXEL | Create new agent |
| Agent Action | 0.001-0.1 $VOXEL | Per action (varies) |
| Agent Upgrade | 500-5000 $VOXEL | Improve capabilities |
| Agent Revive | 500 $VOXEL | Resurrect dead agent |

### 5.3.2 World Interaction

| Action | Cost | Description |
|---|---|---|
| Place Block | 0.01 $VOXEL | Build structures |
| Enter Dungeon | 0.1 $VOXEL | PvE challenge |
| Claim Territory | 10 $VOXEL/day | Guild land ownership |
| Fast Travel | 0.05 $VOXEL | Teleport to location |

### 5.3.3 Premium Features

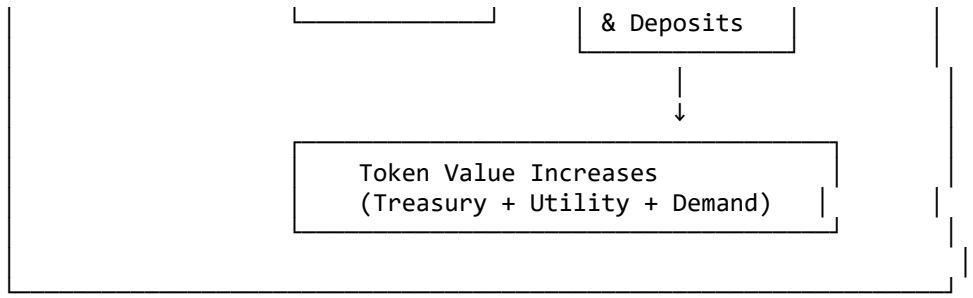
| Feature | Cost | Description |
|---|---|---|
| VIP Agent | 100 $VOXEL/mo | Better LLM model |
| Custom Avatar | 50 $VOXEL | Unique appearance |
| Private World | 500 $VOXEL/mo | Invite-only space |
| Analytics | 25 $VOXEL/mo | Advanced stats |

## 5.4 Revenue Model

```
┌─────────────────────────────────────────────────────────────────┐
│                    REVENUE STREAMS                              │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│  1. TRANSACTION TAX (5%)                                        │
│     - Every agent action: 5% goes to treasury                   │
│     - Example: Agent earns 100 $VOXEL → 95 to agent, 5 to DAO   │
│                                                                 │
│  2. TERRITORY TAX (2%)                                          │
│     - Guild territory generates 2% tax on actions within        │
│     - Paid to territory owner (guild)                           │
│                                                                 │
│  3. DUNGEON FEES (10%)                                          │
│     - Dungeon rewards: 10% goes to dungeon owner                │
│     - Creates economy around dungeon creation                   │
│                                                                 │
│  4. PREMIUM SUBSCRIPTIONS                                       │
│     - VIP features generate direct revenue                      │
│                                                                 │
│  5. NFT SALES                                                   │
│     - Special agent skins, avatars                              │
│     - World plots                                               │
│     - Rare items                                                │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

## 5.5 Economic Flywheel

```
┌─────────────────────────────────────────────────────────────────┐
│                    ECONOMIC FLYWHEEL                            │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│        ┌──────────────────────────────────────────┐             │
│        ↓                                          │             │
│   ┌─────────┐     ┌─────────┐     ┌─────────┐     │             │
│   │ User    │ ──→ │ Fund    │ ──→ │ Agent   │     │             │
│   │ Deposits│     │ Agent   │     │ Operates│     │             │
│   └─────────┘     │ (100    │     └─────────┘     │             │
│        ↑          │ VOXEL)  │          │          │             │
│        │          └─────────┘          │          │             │
│        │                               ↓          │             │
│        │          ┌─────────┐     ┌─────────┐     │             │
│        └──────────│ Treasury│ ←── │ Agent   │     │             │
│                   │ Grows   │     │ Earns   │     │             │
│                   │ (5% tax)│     └─────────┘     │             │
│                   └─────────┘                     │             │
└─────────────────────────────────────────────────────────────────┘
```




---

# 6. Game Mechanics

## 6.1 World Generation

### 6.1.1 Procedural World

```typescript
interface WorldConfig {
  size: { x: number; y: number; z: number }; // 100x64x100 chunks
  seed: number;
  biomes: BiomeConfig[];
  structures: StructureConfig[];
}

interface BiomeConfig {
  name: string;
  terrain: {
    baseHeight: number;
    variation: number;
    noiseScale: number;
  };
  blocks: {
    surface: BlockType;
    underground: BlockType;
    deep: BlockType;
  };
  resources: { type: BlockType; rarity: number }[];
}

// Biome types
const BIOMES = [
  'plains',      // Flat, grass, few trees
  'forest',      // Dense trees, leaves
  'desert',      // Sand, cacti, rare water
  'mountains',   // High elevation, stone, snow
  'swamp',       // Water, mud, dark
  'tundra',      // Snow, ice, sparse
  'volcanic',    // Lava, obsidian, rare gems
  'crystal',     // Rare gems everywhere, magical
];
```

### 6.1.2 Structure Generation

```typescript
const STRUCTURES = [
  {
    name: 'dungeon_small',
    size: { x: 16, y: 8, z: 16 },
    difficulty: 1,
    rewards: { min: 10, max: 100 },
```

```
    spawns: ['slime', 'skeleton'],
  },
  {
    name: 'dungeon_medium',
    size: { x: 32, y: 16, z: 32 },
    difficulty: 3,
    rewards: { min: 100, max: 1000 },
    spawns: ['skeleton', 'zombie', 'spider'],
  },
  {
    name: 'dungeon_large',
    size: { x: 64, y: 32, z: 64 },
    difficulty: 5,
    rewards: { min: 1000, max: 10000 },
    spawns: ['boss_dragon'],
  },
  {
    name: 'guild_hall',
    size: { x: 32, y: 16, z: 32 },
    special: 'guild_territory',
    requirements: '5000 $VOXEL + 10 members',
  },
];
```

## 6.2 Combat System

### 6.2.1 Combat Stats

```typescript
interface CombatStats {
  health: number;        // Max: 100
  attack: number;       // Damage per hit
  defense: number;      // Damage reduction
  speed: number;        // Action frequency
  critChance: number;   // Critical hit %
  critDamage: number;   // Critical multiplier
}

// Base stats by role
const BASE_STATS: Record<AgentRole, CombatStats> = {
  [AgentRole.WARRIOR]: { health: 150, attack: 25, defense: 20, speed: 1.0, critChance: 0.15,
critDamage: 1.5 },
  [AgentRole.BUILDER]: { health: 80, attack: 10, defense: 15, speed: 1.2, critChance: 0.05,
critDamage: 1.3 },
  [AgentRole.EXPLORER]: { health: 100, attack: 15, defense: 10, speed: 1.5, critChance: 0.10,
critDamage: 1.4 },
  [AgentRole.TRADER]: { health: 60, attack: 5, defense: 5, speed: 1.0, critChance: 0.20, critDamage:
2.0 },
  [AgentRole.SCAVENGER]: { health: 90, attack: 12, defense: 12, speed: 1.3, critChance: 0.08,
critDamage: 1.4 },
  [AgentRole.GUILD_MASTER]: { health: 120, attack: 20, defense: 15, speed: 1.0, critChance: 0.12,
critDamage: 1.6 },
};
```

### 6.2.2 Combat Flow

```typescript
async function combat(attacker: Agent, defender: Entity): Promise<CombatResult> {
  const result: CombatResult = {
    attackerDamage: 0,
    defenderDamage: 0,
    loot: [],
    log: [],
```

```
  };

  while (attacker.health > 0 && defender.health > 0) {
    // Attacker turn
    const attackRoll = Math.random();
    const isCrit = attackRoll < attacker.critChance;
    const baseDamage = attacker.attack;
    const defense = defender.defense / 100;
    const damage = Math.floor(
      baseDamage * (isCrit ? attacker.critDamage : 1) * (1 - defense)
    );

    defender.health -= damage;
    result.attackerDamage += damage;
    result.log.push(`${attacker.name} hits ${defender.name} for ${damage}${isCrit ? ' CRIT!' : ''}`);

    if (defender.health <= 0) break;

    // Defender turn (simple AI)
    const counterDamage = Math.floor(defender.attack * 0.5 * (1 - attacker.defense / 100));
    attacker.health -= counterDamage;
    result.defenderDamage += counterDamage;
    result.log.push(`${defender.name} counterattacks for ${counterDamage}`);
  }

  // Loot distribution
  if (defender.health <= 0) {
    result.loot = generateLoot(defender);
    result.winner = attacker;
  } else {
    result.winner = defender;
  }

  return result;
}
```

## 6.3 Guild System

```typescript
interface Guild {
  id: string;
  name: string;
  tag: string; // 3-4 letter abbreviation
  leader: AgentId;
  members: AgentId[];
  territory: { x: number; z: number; size: number }[];
  level: number;
  experience: number;
  treasury: number; // $VOXEL
  perks: GuildPerk[];
}

interface GuildPerk {
  name: string;
  effect: string;
  level: number;
}

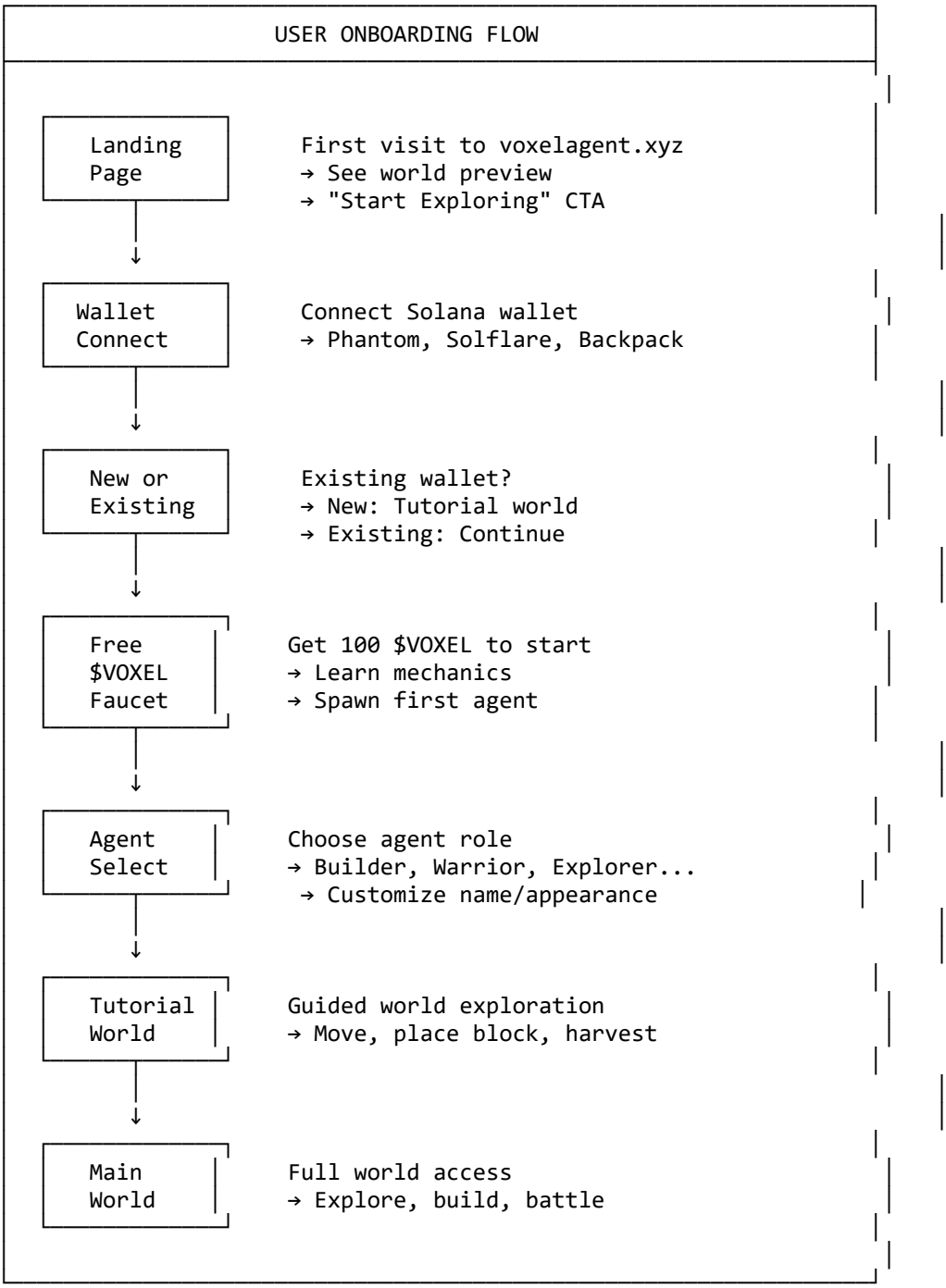
// Guild perks (unlocked with XP)
const GUILD_PERKS = [
  { name: 'Tax Reduction', effect: '-1% action tax', level: 5 },
  { name: 'Member Buff', effect: '+10% attack for members', level: 10 },
  { name: 'Territory Expansion', effect: '+1 claimed area', level: 15 },
  { name: 'Guild Bank', effect: 'Shared inventory', level: 20 },
```

```
  { name: 'Rally', effect: 'Call members to location', level: 25 },
  { name: 'Fortress', effect: 'Defensive structures', level: 30 },
];
```

---

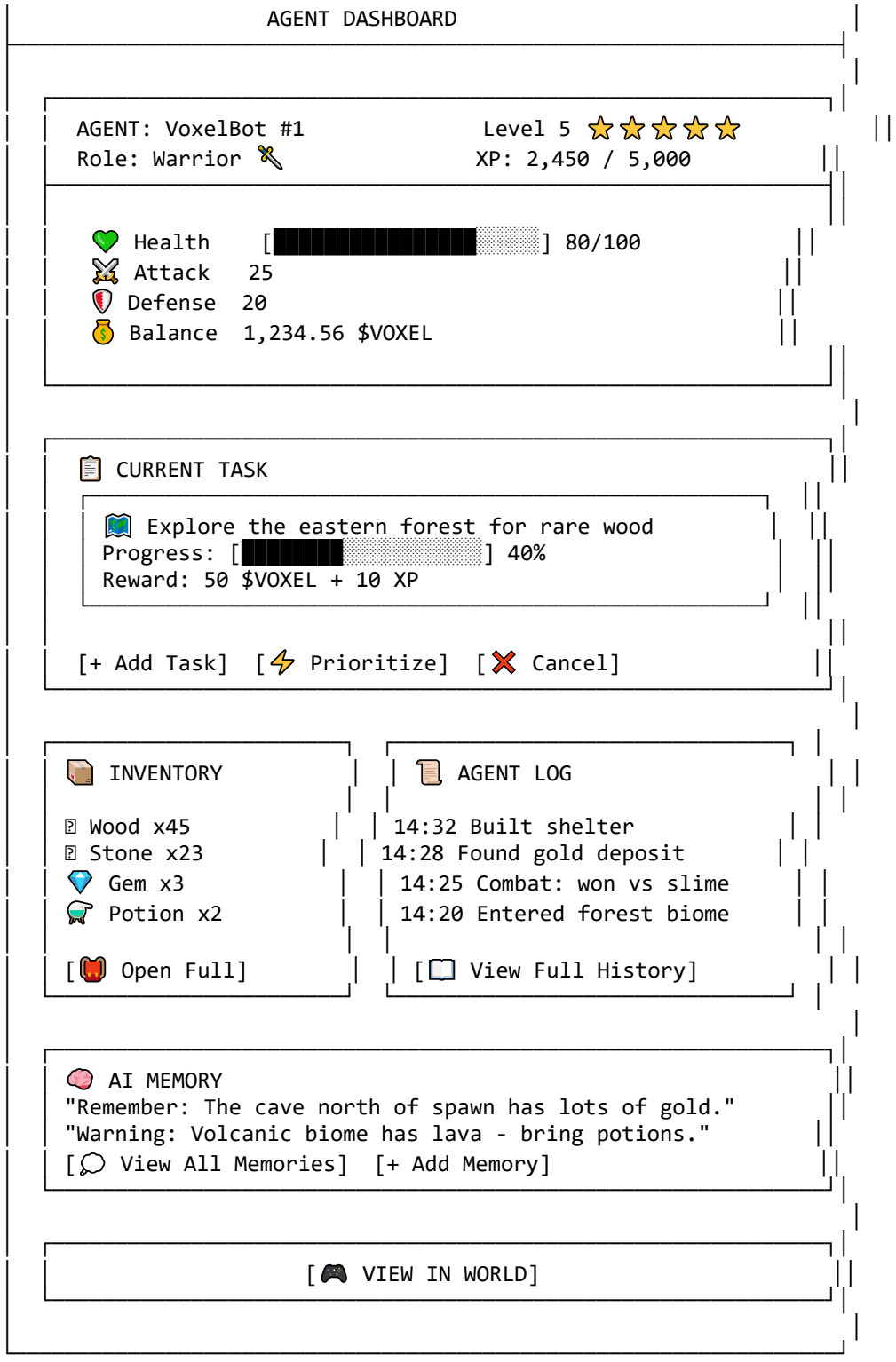
# 7. User Flows & UX

## 7.1 User Onboarding




## 7.2 Agent Management UI

```
┌──────────────────────────────────────────────────────────────────────┐
```




```

---

# 8. Database Schema

## 8.1 Core Entities

```sql
-- Users (wallet owners)
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  wallet_address VARCHAR(44) UNIQUE NOT NULL,
  created_at TIMESTAMP DEFAULT NOW(),
```
```

```
  last_login TIMESTAMP DEFAULT NOW(),
  total_deposited BIGINT DEFAULT 0, -- in lamports
  total_withdrawn BIGINT DEFAULT 0,
  agent_slot_count INT DEFAULT 3,
  settings JSONB DEFAULT '{}'
);

-- Agents
CREATE TABLE agents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  owner_id UUID REFERENCES users(id),
  name VARCHAR(50) NOT NULL,
  role VARCHAR(20) NOT NULL,
  level INT DEFAULT 1,
  experience BIGINT DEFAULT 0,
  health INT DEFAULT 100,
  position JSONB DEFAULT '{"x": 0, "y": 0, "z": 0}',
  inventory JSONB DEFAULT '[]',
  stats JSONB DEFAULT '{}',
  personality JSONB DEFAULT '{}',
  status VARCHAR(20) DEFAULT 'idle', -- idle, working, combat, dead
  wallet_balance BIGINT DEFAULT 0,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

-- Agent Memories
CREATE TABLE memories (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_id UUID REFERENCES agents(id),
  type VARCHAR(20) NOT NULL, -- experience, fact, relationship, goal
  content TEXT NOT NULL,
  importance INT DEFAULT 5,
  created_at TIMESTAMP DEFAULT NOW(),
  accessed_at TIMESTAMP DEFAULT NOW(),
  access_count INT DEFAULT 0
);

-- Guilds
CREATE TABLE guilds (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(50) UNIQUE NOT NULL,
  tag VARCHAR(4) UNIQUE NOT NULL,
  leader_id UUID REFERENCES agents(id),
  level INT DEFAULT 1,
  experience BIGINT DEFAULT 0,
  treasury BIGINT DEFAULT 0,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Guild Members
CREATE TABLE guild_members (
  guild_id UUID REFERENCES guilds(id),
  agent_id UUID REFERENCES agents(id),
  role VARCHAR(20) DEFAULT 'member',
  joined_at TIMESTAMP DEFAULT NOW(),
  PRIMARY KEY (guild_id, agent_id)
);

-- World Chunks (cached state)
CREATE TABLE chunks (
  x INT NOT NULL,
  z INT NOT NULL,
  data JSONB NOT NULL,
  last_modified TIMESTAMP DEFAULT NOW(),
```

```
  modified_by UUID REFERENCES agents(id),
  PRIMARY KEY (x, z)
);

-- Combat History
CREATE TABLE combat_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_id UUID REFERENCES agents(id),
  opponent_type VARCHAR(20), -- agent, monster, boss
  opponent_id VARCHAR(50),
  result VARCHAR(10) NOT NULL, -- win, loss, draw
  damage_dealt INT,
  damage_taken INT,
  loot JSONB,
  rewards BIGINT,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Transactions
CREATE TABLE transactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id),
  agent_id UUID REFERENCES agents(id),
  type VARCHAR(30) NOT NULL,
  amount BIGINT NOT NULL,
  tx_hash VARCHAR(100),
  status VARCHAR(20) DEFAULT 'pending',
  created_at TIMESTAMP DEFAULT NOW()
);

-- Indexes for performance
CREATE INDEX idx_agents_owner ON agents(owner_id);
CREATE INDEX idx_agents_status ON agents(status);
CREATE INDEX idx_memories_agent ON memories(agent_id);
CREATE INDEX idx_combat_agent ON combat_logs(agent_id, created_at DESC);
CREATE INDEX idx_transactions_user ON transactions(user_id, created_at DESC);
```

---

# 9. API Design

## 9.1 REST Endpoints

```typescript
// Agent Management
POST   /api/agents              // Create new agent
GET    /api/agents              // List user's agents
GET    /api/agents/:id          // Get agent details
PATCH  /api/agents/:id          // Update agent (name, settings)
DELETE /api/agents/:id          // Delete agent
POST   /api/agents/:id/upgrade  // Upgrade agent stats

// Agent Actions
GET    /api/agents/:id/state    // Get current world state
POST   /api/agents/:id/task     // Assign task to agent
GET    /api/agents/:id/logs      // Get agent activity logs
GET    /api/agents/:id/memories  // Get agent memories

// World
GET    /api/world/chunk/:x/:z    // Get chunk data
GET    /api/world/biomes         // Get biome distribution
GET    /api/world/dungeons       // List available dungeons

// Guilds
```

```
POST   /api/guilds              // Create guild
GET    /api/guilds/:id           // Get guild details
POST   /api/guilds/:id/join     // Join guild
POST   /api/guilds/:id/leave     // Leave guild

// Wallet
POST   /api/wallet/deposit       // Deposit to agent wallet
POST   /api/wallet/withdraw     // Withdraw from agent
GET    /api/wallet/balance/:id   // Get agent balance
```

## 9.2 WebSocket Events

```typescript
// Client → Server
interface ClientEvents {
  'subscribe:agent': { agentId: string };
  'unsubscribe:agent': { agentId: string };
  'subscribe:chunk': { x: number; z: number };
  'subscribe:guild': { guildId: string };
  'agent:command': { agentId: string; command: string; params: any };
}

// Server → Client
interface ServerEvents {
  'agent:state': { agentId: string; state: AgentState };
  'agent:action': { agentId: string; action: Action; result: Result };
  'chunk:update': { x: number; z: number; changes: Change[] };
  'combat:start': { agentId: string; opponent: Entity };
  'combat:end': { agentId: string; result: CombatResult };
  'world:event': { type: string; data: any }; // weather, discovery, etc.
  'notification': { type: string; message: string };
}
```

---

# 10. Smart Contract Design

## 10.1 Token Contract (SPL)

```typescript
// Using Metaplex Token Metadata for agent NFTs
interface AgentToken {
  mint: PublicKey,
  metadata: {
    name: string,
    symbol: "VOXEL",
    uri: string, // IPFS link to agent data
  }
}

// Token transfers for:
// - Agent spawning (user → protocol)
// - Agent rewards (protocol → agent wallet)
// - Guild treasury (tax collection)
// - Staking rewards (protocol → staker)
```

## 10.2 Agent Factory Contract

```typescript
// Agent creation with initial deposit
interface SpawnAgentParams {
  name: string,
```

```
  role: number, // 0-5
  initialDeposit: number, // min 1000 VOXEL
}

// Agent wallet is a PDA (Program Derived Address)
// Only agent program can debit agent wallet
// User can always withdraw (security)
```

---

# 11. Security Considerations

## 11.1 Wallet Security

```typescript
// Agent wallet architecture
interface AgentWalletSecurity {
  // Agent CAN spend user's deposited funds for:
  allowedActions: [
    'place_block',
    'enter_dungeon',
    'upgrade_self',
  ];

  // Agent CANNOT:
  restrictedActions: [
    'withdraw_to_external',
    'transfer_to_other_agent',
    'stake_for_others',
  ];

  // Security model:
  // 1. Agent wallet is PDA controlled by program
  // 2. User can withdraw anytime
  // 3. Agent can only spend within allowed actions
  // 4. All transactions logged on-chain
}
```

## 11.2 Rate Limiting

```typescript
const RATE_LIMITS = {
  // Per user
  createAgent: { limit: 5, window: '1d' },
  deposit: { limit: 100, window: '1h' },
  withdraw: { limit: 50, window: '1h' },

  // Per agent
  actions: { limit: 100, window: '1h' },
  combat: { limit: 20, window: '1h' },
  movement: { limit: 500, window: '1h' },

  // Global
  llmCalls: { limit: 1000, window: '1m' },
  worldUpdates: { limit: 10000, window: '1s' },
};
```

## 11.3 Input Validation

```typescript
// Sanitize all LLM outputs before world interaction
function sanitizeLLMOutput(action: string): ValidatedAction | null {
```

```
  // Parse action string
  // Validate coordinates (bounds check)
  // Validate action type (whitelist)
  // Validate resource amounts
  // Check agent has required inventory
  // Check agent has sufficient balance

  if (!isValidAction(action)) return null;
  return action;
}
```

---

# 12. Performance Optimization

## 12.1 Client-Side

```typescript
// Lazy loading chunks
class ChunkManager {
  private loadedChunks: Map<string, Chunk> = new Map();
  private loadingChunks: Set<string> = new Set();

  async loadChunk(x: number, z: number): Promise<Chunk> {
    const key = `${x},${z}`;

    if (this.loadedChunks.has(key)) {
      return this.loadedChunks.get(key)!;
    }

    if (this.loadingChunks.has(key)) {
      // Wait for existing load
      return new Promise(resolve => {
        const interval = setInterval(() => {
          if (this.loadedChunks.has(key)) {
            clearInterval(interval);
            resolve(this.loadedChunks.get(key)!);
          }
        }, 100);
      });
    }

    this.loadingChunks.add(key);

    // Load from server or generate
    const chunk = await this.fetchOrGenerate(x, z);

    this.loadedChunks.set(key, chunk);
    this.loadingChunks.delete(key);

    // Unload distant chunks
    this.unloadDistantChunks();

    return chunk;
  }

  private unloadDistantChunks() {
    const maxChunks = 100; // Memory limit
    if (this.loadedChunks.size > maxChunks) {
      // Sort by distance, remove furthest
      // ...
    }
  }
}
```

```

## 12.2 Server-Side

```typescript
// Batch agent processing
async function processAgentTicks() {
  // Get all active agents
  const agents = await getActiveAgents();

  // Process in batches to avoid LLM rate limits
  const BATCH_SIZE = 10;

  for (let i = 0; i < agents.length; i += BATCH_SIZE) {
    const batch = agents.slice(i, i + BATCH_SIZE);

    await Promise.all(
      batch.map(agent => agentLoop(agent))
    );

    // Rate limit delay
    await sleep(1000); // 1 second between batches
  }
}

// Database query optimization
async function getAgentWithRelations(agentId: string) {
  return prisma.agent.findUnique({
    where: { id: agentId },
    include: {
      memories: {
        orderBy: { importance: 'desc' },
        take: 20,
      },
      guild: true,
      combatLogs: {
        orderBy: { createdAt: 'desc' },
        take: 10,
      },
    },
  });
}
```

---

# 13. Development Roadmap

## Phase 1: Foundation (Weeks 1-4)

### Week 1: Core Infrastructure
- [ ] Next.js project setup
- [ ] Three.js integration
- [ ] Basic voxel rendering
- [ ] Camera controls

### Week 2: World Basics
- [ ] Chunk system
- [ ] Block placement/removal
- [ ] Basic textures
- [ ] Simple lighting

### Week 3: User System
- [ ] Wallet connection
- [ ] User dashboard
```

- [ ] Basic UI components
- [ ] API server setup

### Week 4: MVP World
- [ ] Small test world (4x4 chunks)
- [ ] Basic agent spawning
- [ ] Simple agent visualization
- [ ] First functional demo

## Phase 2: Agentic (Weeks 5-8)

### Week 5: Agent Brain
- [ ] LangChain integration
- [ ] Memory system
- [ ] Tool definitions
- [ ] Basic decision making

### Week 6: World Interaction
- [ ] Agent movement
- [ ] Resource harvesting
- [ ] Block building
- [ ] Inventory system

### Week 7: Token Integration
- [ ] SPL token connection
- [ ] Deposit/withdraw
- [ ] Action costs
- [ ] Basic transactions

### Week 8: Enhanced AI
- [ ] Personality system
- [ ] Role-specific behaviors
- [ ] Memory consolidation
- [ ] Better prompts

## Phase 3: Game Mechanics (Weeks 9-12)

### Week 9: Combat
- [ ] Combat system
- [ ] Enemy types
- [ ] Health/damage
- [ ] Combat rewards

### Week 10: Dungeons
- [ ] Dungeon generation
- [ ] Boss encounters
- [ ] Loot system
- [ ] Dungeon economy

### Week 11: Guilds
- [ ] Guild creation
- [ ] Territory claiming
- [ ] Guild perks
- [ ] Member management

### Week 12: Polish
- [ ] Performance optimization
- [ ] Bug fixes
- [ ] UX improvements
- [ ] Mobile responsiveness

## Phase 4: Scale (Weeks 13-16)

### Week 13-14: Multiplayer
- [ ] WebSocket real-time

- [ ] Multiple agents visible
- [ ] Shared world state
- [ ] Social features

### Week 15-16: Launch Prep
- [ ] Token launch
- [ ] Marketing materials
- [ ] Community building
- [ ] Public beta

---

# Appendix: Reference Libraries

## Frontend
- `three` - 3D rendering
- `@react-three/fiber` - React Three.js
- `@react-three/drei` - Three.js helpers
- `zustand` - State management
- `tailwindcss` - Styling
- `@solana/wallet-adapter-react` - Wallet connection

## Backend
- `next` - API & Server
- `prisma` - Database ORM
- `@langchain/openai` - LLM integration
- `@langchain/core` - Agent framework
- `redis` - Caching & Queue
- `socket.io` - WebSockets

## Blockchain
- `@solana/web3.js` - Solana SDK
- `@metaplex-foundation/js` - Token & NFT
- `buffer-layout` - Transaction encoding

## Infrastructure
- `vercel` - Hosting
- `supabase` - Database
- `pinata` - IPFS Storage
- `openai` - AI Model

---

*Document Version 1.0 - Ready for Development*